Code Correctness Is Not in the Text: Five Methods, One Ceiling — Convergent Evidence for CoT Quality Measurement Breakdown in Coding

Anonymous Author(s)
Affiliation
Address
email

Abstract

Can the quality of a coding solution be read from its chain-of-thought (CoT) trace? We test this question using five methods on the same feature family—token-level uncertainty signals and text-trajectory statistics—across three domains: math competitions, science (GPQA), and coding (LiveCodeBench-v5). All experiments use a single model, **DeepSeek-R1-0528-Qwen3-8B**.

In math and science, the answer is clearly *yes*: a simple interpretable probe achieves AUC-of-AUROC 0.958 and 0.799 respectively. The signal is temporally structured and transfers across early-stopping anchor positions. A best-of-$N = 64$ reranking experiment delivers $+10.0\%$ and $+8.0\%$ pass@1 gains.

In coding, the answer is consistently *no*. (1) The same probe yields AUC-of-AUROC 0.434, *below* a single-feature token-confidence baseline. (2) A sweep of 83 coding-specific scalars peaks at AUROC 0.556. (3) A nonlinear MLP classifier does not escape the ceiling. (4) Self-supervised pre-training on 42,000 unlabeled CoT traces—including a carefully engineered domain-specific variant with VICReg anti-collapse and five-term structured loss—tops out near 0.50 AUROC for coding at full labels. (5) A token-level surface-noise ablation (de-knotting) with all per-token signal arrays properly masked shows that in math, knot tokens *carry* the discriminative signal (ΔAUROC $= -0.049$ after removal), while in coding, no masking strategy recovers signal.

These five methods converge on a single explanation: code correctness depends on executable state evolution, and this information is not encoded in token trajectories. No amount of data, no change of algorithm, and no surface cleaning recovers a signal that is absent from the feature family. The failure is a *measurement* failure, not a modeling failure.

1 Introduction

Process supervision and verifier-based reranking rest on a shared assumption: that chain-of-thought traces encode answer quality, and that this quality signal can be extracted by a probe of reasonable complexity. This is well-established for mathematical reasoning [3–5]. The open question is whether the same is true across domains.

This paper is not about building a better verifier. It is about *measuring* whether a specific feature family encodes correctness in each domain—and about accumulating enough evidence to be confident about a null result in coding.

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

Our starting point is deliberately simple: five hand-specified features (three trajectory statistics, one token-confidence summary, and one neuron-consistency mean), a `StandardScaler` $\rightarrow$ `LogisticRegression` pipeline, evaluated at anchor positions $\{10, 40, 70, 100\}\%$ of the visible trace. Simple enough that a performance failure can be attributed to the features, not to the model. We then expand: 83 coding-side scalars, a nonlinear MLP, self-supervised pre-training with up to 42,000 unlabeled runs, and a token-level ablation study.

The central finding is a **convergent measurement breakdown**. Every method applied to the token-trajectory feature family reaches the same ceiling near AUROC 0.5 for coding correctness. In contrast, math and science show strong, temporally structured, transferable signals from the same features. The gap is not reducible to label scarcity, model capacity, or surface noise. The most parsimonious account is that code correctness depends on executable state evolution—an object that text-surface features observe only as an imperfect shadow.

A downstream best-of-$N = 64$ reranking experiment confirms the practical consequence: the probe gains $+10.0\%$ pass@1 on math and $+8.0\%$ on science, but -0.6% (effectively zero) on coding.

2 Related Work

Process supervision and verifiers. Learning outcome reward models [3] and process reward models [4] established that step-level supervision improves over outcome-level alone. Math-Shepherd [5] automated this with LLM annotation; ProcessBench [6] benchmarks step-level error detection; generative verifier formulations [7] frame reward modeling as next-token prediction. Our work does not compete with any of these—it asks whether simpler, cheaper features measure the same thing across domains.

CoT faithfulness and self-correction. Whether CoT traces faithfully reflect model reasoning is contested [8]. Intrinsic self-correction work [9, 12] documents that reflective text does not always improve reasoning; self-refine [10] and reflexion [11] show that iterative revision helps only with external feedback. We add a cross-domain dimension: not only does self-correction fail to reliably improve reasoning, but the *measurability* of reasoning quality from text trajectory is itself domain-conditioned.

Self-supervised and semi-supervised probing. Self-supervised learning for representation quality has been studied in NLP [13], but its application to CoT trajectory probing is unexplored. We evaluate SSL pre-training directly as a label-efficiency tool—if the features encode something latent about correctness, SSL should help at low label fractions. The finding that SSL tops out near AUROC 0.5 for coding at full labels rules out "need more labeled data" as the bottleneck.

Measurement invariance. The psychometric literature on measurement invariance [14] establishes that the same instrument can measure different constructs in different populations. Probing work in NLP [15] makes a related point. We extend this logic to verifier settings: a feature family that measures correctness in math may observe a qualitatively different object in coding.

3 Setup

3.1 Model, domains, and data

All CoT traces are generated by **DeepSeek-R1-0528-Qwen3-8B** at temperature 1.0. **Math:** 7,680 aligned runs across AIME24, AIME25, BRUMO25, and HMMT25 (competition math; ground-truth binary correctness). **Science:** 12,672 runs from GPQA (graduate-level science MCQ). **Coding:** 10,688 runs from LiveCodeBench-v5 (competitive programming; judge-verified correctness).

3.2 Feature family

We use five hand-specified features computed from cached token streams and neuron activations. These are kept intentionally cheap so that failure is attributable to the feature family, not to engineering.

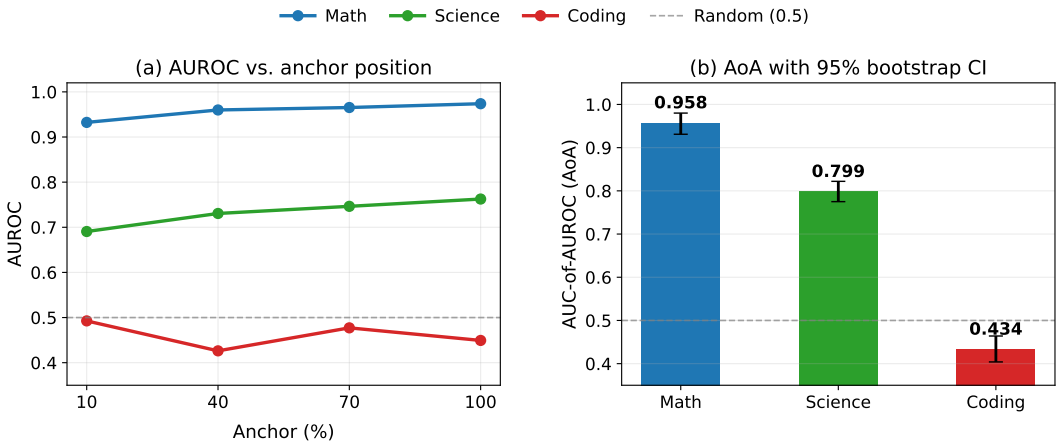

Figure 1: **(a)** AUROC at each anchor position (10%–100% of trace visible) for the three domains. Math and science rise with more context; coding oscillates near 0.5 regardless of how much trace is revealed. **(b)** AUC-of-AUROC (AoA) with 95% bootstrap CIs. The domain gap is large and statistically well-separated.

Token-level signals. `tok_conf` is the mean negative log probability over visible prefix tokens (per-token token confidence / entropy). Four additional arrays are cached per token: `tok_gini`, `tok_logprob`, `tok_neg_entropy`, `tok_selfcert`. All five are available per-token and used in ablation studies.

Trajectory statistics. `traj_continuity`: mean Jaccard similarity between consecutive 32-token activation slices. `traj_novelty`: mean $1 - \max_j \text{Jaccard}(s_i, s_j)$ over all prior slices. `traj_reflection_count`: negated count of non-adjacent slice revisits with Jaccard > 0.30.

Neuron summary. `nc_mean`: mean row width of the sparse activation bank across the visible prefix.

3.3 Probe pipeline and metrics

The base probe is `StandardScaler` $\rightarrow$ `TruncatedSVD` $\rightarrow$ `LogisticRegression`, fit separately per domain. Evaluation uses leave-one-problem-out cross-validation (GroupKFold by `problem_id`).

The primary metric is **AUC-of-AUROC (AoA)**: area under the anchor-wise AUROC curve at positions $\{10, 40, 70, 100\}\%$, used as an early-stopping summary. Bootstrap 95% CIs on AoA use $B = 10{,}000$ resamplings of datasets within each domain (7 for math, 2 for science).

4 Math and Science: Strong, Temporally Structured, Transferable

Figure 1 and Table 1 summarize the main probe result. Math achieves AoA 0.958 [0.931, 0.980] and AUROC 0.982 at the full trace. Science reaches AoA 0.799 [0.775, 0.822] and AUROC 0.841. Coding falls to AoA 0.434 and AUROC 0.407—below the token-confidence baseline (0.506).

Temporal structure in math. The math signal is not a final-score story—it is a *process* story. The Spearman correlation between `traj_reflection_count` and correctness deepens as more of the trace is revealed:

$$\rho_{10\%} = -0.306, \quad \rho_{40\%} = -0.383, \quad \rho_{100\%} = -0.491.$$

Early reflection counts encode something real, but late reflection counts encode failure more strongly. This is the signature of a process feature: it measures whether the model converged, not just what it said at the end.

Frozen-basis transfer. We freeze the low-rank basis learned at one anchor and refit only the logistic head at another: $\Delta\text{AUROC} = \text{AUROC}_{\text{frozen}} - \text{AUROC}_{\text{task-specific}}$. For math, forward and backward gaps are both near -0.001 to -0.006, indicating that the feature geometry is stable across anchor

Domain	AoA	A100	Fwd Δ	Bwd Δ
Math	**.958** [.931, .980]	**.982**	$-.001$	$-.006$
Science	**.799** [.775, .822]	**.841**	$-.010$	$-.001$
Coding	.434 [.404, .464]	.407	—	—
Baselines				
`tok_conf`	.506/.809/.506	.490/.856/.490	—	—
SC oracle[†]	.953/.921/.961	.953/.921/.961	—	—

Table 1: Main holdout result. AoA = AUC-of-AUROC; values in brackets are 95% CIs from per-dataset bootstrap ($B = 10{,}000$). A100 = AUROC at the 100% anchor. Frozen-basis ΔAUROC: Fwd = source-to-target transfer gap; Bwd = target-to-source. Baseline rows report math/science/coding values. [†]SC oracle = self-consistency upper bound (requires ground truth).

positions. Science has a larger forward gap (-0.010), suggesting reasoning maturity is less uniformly structured. Coding has no meaningful forward transfer to report because the within-domain AUROC is already near chance.

MLP sanity check. Replacing the linear head with a one-hidden-layer MLP under five-fold GroupKFold gives $0.940/0.737/0.499$ mean-anchor AUROC for math/science/coding, and $0.965/0.760/0.507$ at the 100% anchor. The cross-domain pattern is not a linear-head artifact.

5 Coding: Five Methods, One Ceiling

We apply five independent methods to ask whether anything in the token-trajectory feature family encodes coding correctness. All five reach the same answer.

5.1 Method 1: Full-scale interpretable probe

On the 10,688-run LiveCodeBench holdout table, the best pairwise proxy for self-corrective control-flow turbulence—`condition_reversal_per1k`—is statistically nonzero ($\rho = -0.062$, $p = 1.15 \times 10^{-10}$). But the core probe achieves AoA 0.434, below the token-confidence baseline (0.506). The feature family carries the faintest trace of signal—but not enough to separate correct from incorrect runs usefully.

5.2 Method 2: Broader feature sweep (83 scalars)

We screen 83 interpretable coding-side scalars: struct complexity counts, control-flow features, comment density, CoT-trace length, code-trace alignment scores, and token-level statistics. The strongest negative scalar is `struct_density` ($\rho = -0.110$, AUROC 0.565), followed by `cot_trace_count` ($\rho = -0.099$, AUROC 0.558). The strongest program-structure count `cf_n_ifs` has $\rho = +0.109$ (AUROC 0.563): more complex code is weakly *more* correct, not less. Combining the top four interpretable scalars reaches only AUROC 0.556 under GroupKFold. No scalar or combination approaches the math probe level.

5.3 Method 3: Nonlinear classifier (MLP)

A one-hidden-layer MLP on the full 25-dimensional feature cache gives AUROC 0.499/0.507 at mean-anchor and 100%-anchor respectively. Changing the classifier does not rescue coding. The ceiling is not a linear-model artifact.

5.4 Method 4: Self-supervised pre-training

Motivation. Methods 1–3 use labeled data exclusively. Method 4 asks a different question: if we train an encoder on all 42,000 *unlabeled* CoT runs, can we learn a latent space where coding correctness becomes separable? If the answer were yes, it would imply that correctness information

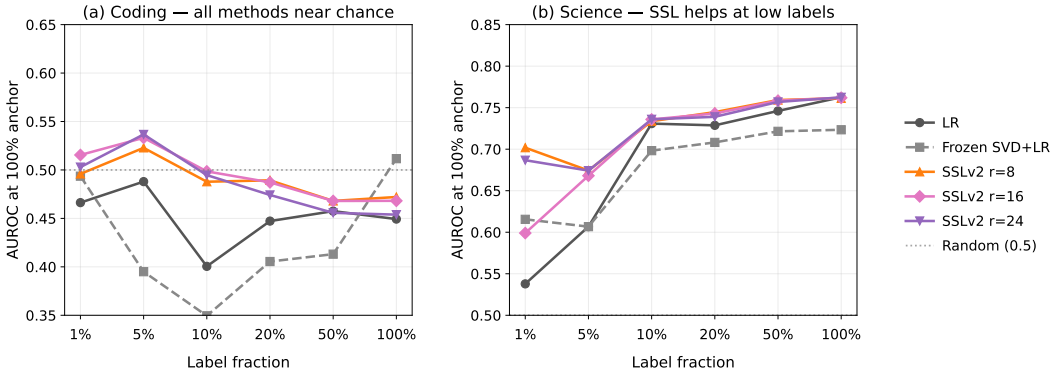

Figure 2: **SSL ceiling in coding vs. science. (a)** In coding, all SSL variants and baselines cluster near AUROC ≈ 0.45–0.55 across all label fractions. Even the best-engineered SSL (ssl_v2_r24, VICReg, domain-specific encoder, 500 epochs) does not escape the ceiling at full labels. **(b)** In science, SSL v2 provides a meaningful $+6.7$pp gain at low label fractions, confirming that SSL can exploit latent structure when that structure exists. The contrast rules out label scarcity as the bottleneck in coding.

is encoded in the feature space but is hard to extract with limited labeled data. If the answer is no, it implies the feature space simply does not contain the information.

SSL v1: shared cross-domain basis. We train a masked-reconstruction SSL encoder on all 42,000 runs with ranks $r \in \{4, 8, 16\}$, using three loss terms: masked feature recovery, cross-anchor alignment, and raw/rank view consistency. Results:

- $r = 8$: universal collapse to AUROC $= 0.5000$ on all 18 evaluation rows. The combined loss saturated at ≈ 0.432 before epoch 60 and did not decrease through epoch 300. All samples collapsed to the same point in $\mathbb{R}^8$.
- $r = 16$: partial recovery (AUROC 0.779 on math, 0.502 on coding at 100% labels). Still well below the `no_svd_lr` baseline.
- No rank improved label efficiency in coding. At 1% labels, `no_svd_lr` already outperforms or matches the best SSL variant at most fractions.

Failure modes: cross-domain pollution (math structure diluted by coding), reconstruction task too easy for linear autoencoder, no LR schedule to escape early saddle.

SSL v2: domain-specific structured SSL. We redesign the SSL pipeline to address v1's failures: (1) separate encoder per domain, (2) five-term structured loss (family-block masking + NT-Xent contrastive + future-anchor prediction + raw/rank consistency + VICReg anti-collapse), (3) cosine warm-restart LR schedule, (4) up to 500 epochs. All 27 trained bundles (3 domains $\times$ 3 ranks $\times$ 3 seeds) achieve zero dead dimensions. Loss decreases by multiple orders of magnitude (math $r = 8$: 325k $\rightarrow$ 5k; coding $r = 24$: 208k $\rightarrow$ 60).

Figure 2 shows the coding ceiling. Despite best-engineering SSL, coding at full labels (lf $= 1.0$) reaches AUROC ≈ 0.45–0.47—*below* the simple `no_svd_lr` baseline (0.449). At low label fractions (lf $= 0.05$), SSL v2 provides a modest gain (ssl_v2_r24 reaches 0.537 vs. no_svd_lr 0.488, $+4.9$pp), consistent with a regularization effect in the low-data regime. Science shows a larger SSL benefit ($+6.7$pp at lf=0.05), consistent with the domain where features do carry signal.

Interpretation. The SSL v2 result is the key experiment in this section. If there were latent structure in the coding feature space that correlates with correctness—even structure that linear probing cannot access with limited labels—SSL pre-training on 42,000 runs should find it. It does not. The training loss decreases (the encoder is learning *something*), but what it learns is not predictive of correctness. The most parsimonious account: the token-trajectory features do not encode execution correctness, not because of data or model limitations, but because the two quantities are causally decoupled at the feature level.

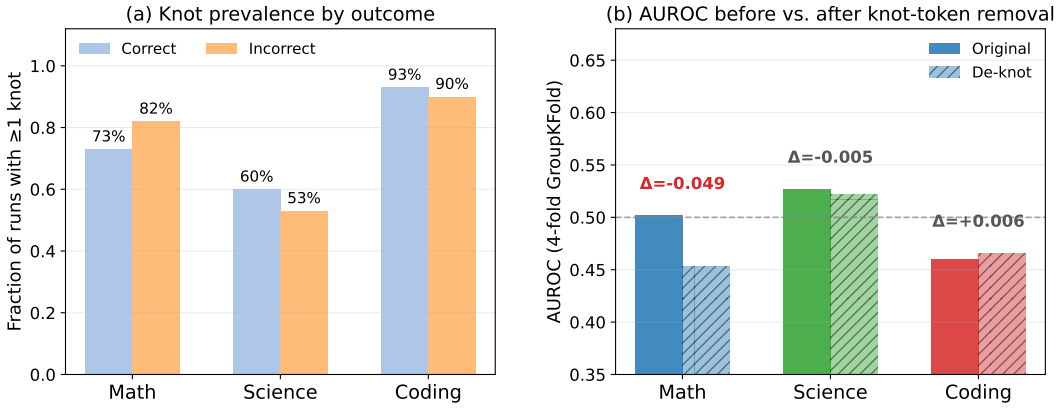

Figure 3: **De-knotting ablation across all three domains. (a)** Knot prevalence differs by domain and by outcome. In math, knots are more prevalent in incorrect runs (82% vs. 73%)—the expected pattern if knots are failure indicators. In science, the difference is small and reversed (53% incorrect vs. 60% correct). In coding, knots are near-symmetric (90% vs. 93%). **(b)** Removing knot token spans from all five per-token signal arrays *hurts* math AUROC ($\Delta = -0.049$) and leaves science and coding neutral. The math result confirms that knot tokens are the discriminative signal itself. The coding neutral result confirms that the failure is structural.

5.5 Method 5: Token-level surface-noise ablation (de-knotting)

A complementary hypothesis is that "knot" spans—circular reasoning segments where the model revisits the same idea without progress—add noise to the token stream, masking an otherwise-functional signal. If true, removing knot tokens should improve AUROC. We test this directly using a proper token-level ablation.

Protocol. GLM-4-Flash (with domain-specific prompts) identifies knot character spans in $n = 200$ balanced traces per domain (100 correct, 100 incorrect). We then map spans to token indices via the fast tokenizer's offset mapping and remove the corresponding positions from *all five* per-token cached arrays (`tok_conf`, `tok_gini`, `tok_logprob`, `tok_neg_entropy`, `tok_selfcert`), as well as splicing the text. This is a full token-level ablation, not a text-only operation.

Results. Figure 3 shows knot prevalence and the AUROC response to removal.

- **Math**: 155/200 runs knotted; incorrect 82% vs. correct 73%. Removing all knot tokens *hurts* AUROC from 0.502 to 0.453 ($\Delta = -0.049$). The knot-associated spans are the discriminative signal itself—they encode the failure pattern (persistent circular reasoning in incorrect runs). Removing them degrades the verifier.
- **Science**: 113/200 knotted; correct 60% vs. incorrect 53% (slight reversal). Removal neutral: AUROC $0.527 \rightarrow 0.521$ ($\Delta = -0.006$). Knot tokens carry near-zero signal.
- **Coding**: 183/200 knotted; near-symmetric (93% vs. 90%). Removal neutral: AUROC $0.460 \rightarrow 0.466$ ($\Delta = +0.006$). No masking strategy recovers signal, in any direction.

The math result is especially important: it rules out the interpretation that "verifiers work in math because the knot spans there are informative." Rather, the knot spans in math *are* the informative spans—removing them confirms it. The coding null result confirms that the feature-space failure is not surface noise.

5.6 Convergence summary

Table 2 collects all five methods' results for coding.

Method	Best coding AUROC	Verdict
Interpretable probe (LogReg)	0.434 (AoA) / 0.407	Below baseline
Feature sweep (83 scalars)	0.556 (top-4 combo)	Marginal
Nonlinear MLP	0.499–0.507	No change
SSL v1 (cross-domain)	0.502 ($r = 16$, 100% lf)	Collapse; no gain
SSL v2 (domain-specific, VICReg)	0.537 (lf=0.05) / 0.454 (lf=1.0)	Low-label reg. only
De-knotting (token-level mask)	$\Delta = +0.006$	Neutral

Table 2: All five methods applied to coding correctness prediction. No method escapes AUROC ≈ 0.5 at full labels. SSL v2 shows a small regularization benefit at low label fractions but does not change the full-label ceiling.

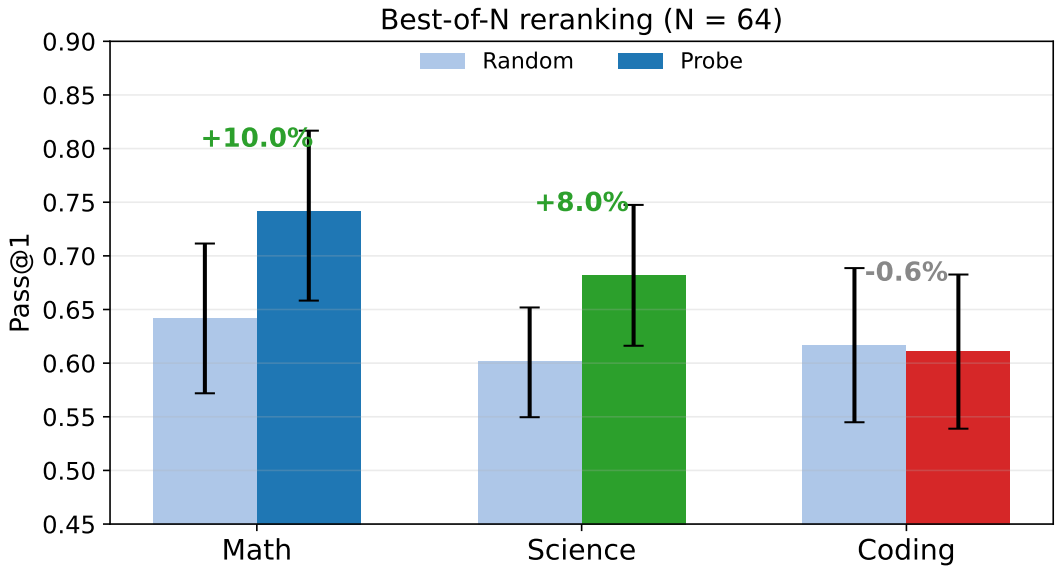

Figure 4: Best-of-$N = 64$ reranking. Probe reranking gains $+10.0\%$ on math and $+8.0\%$ on science, while yielding -0.6% (effectively zero) on coding. Error bars are 95% bootstrap CIs from problem resampling ($B = 10{,}000$, $n \geq 120$ problems per domain).

6 Downstream Consequence: Best-of-N Reranking

The AUROC failure translates directly into a practical consequence. We run a best-of-N selection experiment with $N = 64$ candidates per problem: the run with the highest probe score at 100% anchor is selected, and we report pass@1. The baseline is the per-problem mean accuracy (expected pass@1 under random selection).

Table 3 and Figure 4 show the result. Math gains $+10.0$ percentage points, science gains $+8.0$ pp, and coding gains effectively nothing (-0.6 pp; 95% CIs overlap completely). The measurement breakdown has a direct practical cost: deploying this probe for best-of-N selection in coding wastes the selection budget and provides no lift.

We do not claim that all verifiers fail on coding. Code-execution-aware verifiers, models with access to compiler feedback, or probes over hidden states have different information. The finding is specific to this feature family.

Domain	n	Random	Probe	Δ
Math	120	.642 [.572, .712]	.742 [.658, .817]	**+.100**
Science	198	.602 [.550, .652]	.682 [.616, .748]	**+.080**
Coding	167	.617 [.545, .689]	.611 [.539, .683]	−.006

Table 3: Best-of-N reranking pass@1 ($N = 64$). Values in brackets are 95% bootstrap CI ($B = 10{,}000$). Coding pass@1 is statistically indistinguishable from random.

7 Why the Feature Family Fails in Coding

The key asymmetry. Math reasoning produces process-level evidence: when a solver is going in circles, the token stream says so—reflection counts increase, continuity drops, entropy oscillates. These patterns are causally downstream of the failure. In coding, the critical variable is *whether the code compiles and passes tests*. A beautiful, well-organized CoT can produce incorrect code; a chaotic, uncertain CoT can produce correct code. The text is causally upstream of the code, but correctness is determined downstream of execution.

Why SSL does not help. Self-supervised learning on CoT trajectories can learn structure that exists in those trajectories—temporal consistency, within-problem coherence, cross-anchor alignment. What it cannot do is discover a correlation between trajectory structure and correctness if that correlation does not exist in the data. The SSL result confirms this: loss goes down (structure is learned), but AUROC stays near 0.5 (correctness is not part of the learned structure).

Domain-conditioning as measurement invariance. The psychometric concept of *measurement invariance* [14] asks whether an instrument measures the same latent construct across groups. Our results suggest a measurement non-invariance: `traj_reflection_count` measures "failure to converge" in math (where convergence is defined by a closed-form answer) but measures "exploratory effort" in coding (where convergence requires successful execution). Same feature name, different construct.

What would fix coding. Three directions are worth pursuing: (1) *Execution-aware features*: AST depth, cyclomatic complexity, or compiler-feedback signals that track executable state rather than text surface. (2) *Hidden-state probes*: intermediate activations may encode execution semantics not visible in the token stream. (3) *Multi-model replication*: separating model-specific effects from domain-specific effects requires testing the same features on a second model with different coding architecture.

8 Limitations and Conclusion

Limitations. First, all experiments use a single model (DeepSeek-R1-0528-Qwen3-8B). The ceiling may differ for models with different reasoning styles. Second, the SSL experiments use up to 42,000 unlabeled runs—substantial but not exhaustive; at dramatically larger scale the picture may change. Third, the RL regime is excluded; verifier drift under RL fine-tuning is a different axis (regime-conditioning rather than domain-conditioning) and is studied separately.

Broader impacts. A reliable estimate of when CoT-based quality measurement fails is useful because it can prevent misplaced trust in verifier-driven ranking systems. The positive impact is methodological caution: our results help practitioners avoid deploying text-only correctness proxies in coding settings where they provide no lift. The negative side is that better characterization of verifier failure modes could be reused to bypass lightweight quality-control systems; for that reason, we frame the result as an argument for execution-aware safeguards rather than for more aggressive test-time ranking over code alone.

Conclusion. We have tested five methods on the question of whether token-trajectory features encode coding correctness in CoT traces. All five converge on no:

Within this feature family, code correctness is not in the text. Simple probes, broad feature sweeps, nonlinear classifiers, self-supervised pre-training on 42,000 unlabeled runs, and token-level noise ablation all return the same result: coding AUROC cannot be meaningfully lifted above chance. The most informative negative result is the SSL experiment—if latent structure existed in these features, SSL would find it. It does not. The contrast with math (AoA 0.958, $+10\%$ reranking gain) and science (AoA 0.799, $+8\%$ gain) makes the specificity of the coding failure clear. Before deploying a CoT-based verifier in a new domain, the right question is not "what is the training accuracy?" but "does this feature family observe the right causal object?" In coding, it does not.

Artifacts. Quantitative claims reproduce from `results/tables/`. Figure generation for this revision is in `scripts/gen_figures_v12_5.py`. Paper source is in `workshop/cotknot/paper/`.

References

[1] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[2] Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

[3] Karl Cobbe et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

[4] Hunter Lightman et al. Let's verify step by step. *arXiv:2305.20050*, 2023.

[5] Peiyi Wang et al. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *ACL*, 2024.

[6] Chujie Zheng et al. ProcessBench: Identifying process errors in mathematical reasoning. In *ACL*, 2025.

[7] Lunjun Zhang et al. Generative verifiers: Reward modeling as next-token prediction. In *ICLR*, 2025.

[8] Tamas K. Feher de Lanham et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv:2307.13702*, 2023.

[9] Dancheng Liu et al. Large language models have intrinsic self-correction ability. In *NeurIPS*, 2024.

[10] Aman Madaan et al. Self-Refine: Iterative refinement with self-feedback. *arXiv:2303.17651*, 2023.

[11] Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv:2303.11366*, 2023.

[12] Jie Huang et al. Large language models cannot self-correct reasoning yet. *arXiv:2310.01798*, 2023.

[13] Jacob Devlin et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[14] Robert E. Vandenberg and Charles E. Lance. A review and synthesis of the measurement invariance literature. *Organizational Research Methods*, 3(1):4–70, 2000.

[15] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the paper's central claim as a domain-conditional measurement result for token-trajectory features, and the later experiments in Sections 4–7 match that scope.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 explicitly lists the major limitations, including the single-model scope, finite SSL scale, and exclusion of the RL regime.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper is empirical and does not present new theorems or formal proofs.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3–6 specify the model, datasets, feature family, cross-validation setup, anchor positions, metrics, SSL settings, and evaluation protocols; the artifact paragraph points to the exact scripts and result tables used for figure generation.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The paper source, figure-generation code, and result tables are organized in the repository, but the full anonymized release package for caches, model access, and all external assets is not yet bundled as an open supplemental artifact.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: The setup section and the method subsections specify data domains, split protocol, anchor grid, feature definitions, probe architecture, SSL objectives, label fractions, and evaluation metrics, which are sufficient to understand the reported comparisons.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The main tables and figures report 95% bootstrap confidence intervals, and Section 3 defines the bootstrap procedure and resampling units used for AoA and reranking summaries.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The present draft does not yet include a full hardware, memory, and runtime accounting for each experiment family; this information should be added in the supplemental reproducibility package.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work evaluates benchmark traces and model outputs without human-subject intervention, and we do not identify any deviation from the NeurIPS Code of Ethics in the research process described here.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 now includes a broader-impacts paragraph that discusses both the positive value of avoiding misplaced verifier trust and the possible negative use of failure-mode characterization to bypass lightweight controls.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: The paper does not release a new high-risk generative model or scraped dataset; it analyzes existing benchmark traces and reports aggregate evaluation results.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: The draft cites the model and benchmark sources, but it does not yet provide an explicit per-asset license and terms-of-use summary in the paper or supplemental package.

Guidelines:

- The answer NA means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [NA]

    Justification: The paper's contribution is an empirical analysis and does not introduce a new public dataset, pretrained model, or benchmark as a primary research artifact.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

Justification: The paper does not involve human subjects or crowdsourced participants, so IRB review is not applicable.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Sections 3 and 5 explicitly state that the study analyzes CoT traces generated by DeepSeek-R1-0528-Qwen3-8B and uses GLM-4-Flash for knot-span annotation in the de-knotting protocol.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.